

Make $\pi_{0.5}$ Learn to Call Astra for Reliable Robot Manipulation

Jianli Sun^{1,*} Chengxiang Li^{2,*} Yonglin Tian¹

¹Institute of Automation, Chinese Academy of Sciences

²School of Information and Intelligent Engineering, University of Sanya

Abstract

GPT-6 Astra provides robust embodied reasoning, but frequent external queries increase inference time and token costs. We equip a frozen $\pi_{0.5}$ policy with a lightweight CALL head that learns to predict when assistance is necessary. CALL reuses features from the standard policy forward pass and operates locally without consulting external models. Astra is activated solely for critical recovery events and returns control to $\pi_{0.5}$ upon stabilization. Evaluated across 40 LIBERO tasks (200 episodes), our system achieves a 97.5% success rate, outperforming the 95.5% baseline of pure $\pi_{0.5}$. In the Object, Goal, and Spatial suites, Astra executes only 5.49% of environment steps, consuming approximately 5,027 tokens per test episode. This selective intervention preserves the fast ordinary execution of $\pi_{0.5}$ while seamlessly integrating Astra’s reasoning on demand.

1 Introduction

General-purpose reasoning models are increasingly capable robot decision makers. Recent evaluations show that GPT-6 Astra achieves high success rates on real-robot manipulation tasks [1]. Furthermore, combining Astra’s visual reasoning with $\pi_{0.5}$ by continuously reviewing and correcting policy proposals significantly improves hybrid success rates over direct Astra control [2]. These results highlight the value of Astra’s reasoning for embodied manipulation.

However, this capability incurs substantial inference latency and token expenditure. Continuous external review during the policy-proposal cycle [2] introduces overhead even when the underlying proposal is correct. Consequently, frequent consultation slows action execution and wastes computational resources on routines a learned policy can handle independently.

We augment a frozen $\pi_{0.5}$ policy [3, 4] with a lightweight CALL detector that predicts the need for assistance during inference. The detector operates locally using the policy’s existing representations, avoiding external model queries. A temporal trigger transfers control to Astra, which executes recovery actions or returns control to $\pi_{0.5}$. Once stabilized, the learned policy resumes as the active controller.

Our method integrates learned assistance prediction, selective handoff, and closed-loop recovery. Experiments on four LIBERO suites [7] evaluate task success, intervention rates, token usage, and action throughput. The results demonstrate that a fast learned policy can efficiently leverage Astra’s reasoning precisely when needed, without compromising baseline execution speeds.

2 Method

2.1 CALL detector architecture

We use the frozen OpenPI `pi05_libero` checkpoint as the base policy. A feature hook extracts representations from its 10×7 action-chunk proposal. The detector input is

$$x = [h_{\text{sem}}; h_{\text{act}}; u], \quad \dim(x) = 2048 + 1024 + 44 = 3116. \quad (1)$$

The semantic feature h_{sem} is a masked mean of the final vision/language-prefix representation. The action feature h_{act} pools the final action-token representation. The auxiliary vector u combines eight robot-state values with 36 action-chunk statistics: per-dimension means and standard deviations, the flattened chunk norm, the first and last actions, and their difference.

Table 1: CALL-head branches. Each projection is a linear layer followed by GELU.

Branch	Input dimension	Projection dimension
Semantic representation	2,048	128
Action representation	1,024	128
Robot state and action statistics	44	64
Concatenated projection		320

The concatenated projection passes through LayerNorm, a 256-unit linear layer, GELU, LayerNorm, and a scalar output. The CALL head contains 479,937 trainable parameters. Feature preprocessing and temperature calibration produce the help score

$$p_{\text{help}} = \sigma(z_{\theta}(x)/T). \quad (2)$$

CALL shares the existing policy forward pass and computes the help score locally from policy features and robot/action statistics. Detection requires no additional model invocation. Astra enters the control loop only when the help score triggers a handoff.

2.2 Training data and competence supervision

We source failure-state training data from the **LIBERO-Recovery dataset** [5, 6], utilizing its task scenes, saved simulator states, and pre-failure trajectories. We combine these with normally progressing $\pi_{0.5}$ states to train assistance prediction across both complex and routine executions.

For each state, $\pi_{0.5}$ continuation trials estimate the probability of successful completion. If K of M trials succeed, the competence estimate and help target are

$$q_{\pi} = K/M, \quad y_{\text{help}} = 1 - q_{\pi}. \quad (3)$$

The help target increases as continuation competence decreases. We train the head using soft binary cross-entropy against y_{help} . Low- and high-competence groups use $q_{\pi} \leq 0.2$ and $q_{\pi} \geq 0.8$, respectively; intermediate states retain their soft targets.

We split the data by task into training, early stopping, calibration, and testing sets. Operating thresholds and temperature are calibrated prior to evaluation. Table 2 summarizes the 5,055 competence-state examples across 37 held-out task folds.

We deploy 37 task-specific CALL heads and two suite-level fallback heads. At runtime, the system defaults to the task-specific head if available, falling back to the suite-level head otherwise. Each head operates with calibrated ordinary and hard trigger thresholds.

Table 2: Offline competence-supervision data across task folds.

Suite	Held-out task folds	Competence-state examples
LIBERO-10	10	2,150
Object	10	1,625
Goal	8	860
Spatial	9	420
Total	37	5,055

2.3 Handoff between $\pi_{0.5}$ and Astra

The system evaluates CALL at each policy query, consuming five actions from the policy chunk between queries during normal execution. A five-query score window confirms sustained help predictions: at least four scores must exceed the calibrated threshold. A hard threshold triggers immediate handoff regardless of the cooldown period:

$$\text{CALL}_t = [p_t \geq \tau_{\text{hard}}] \vee \left[\sum_{j=0}^4 \mathbf{1}(p_{t-j} \geq \tau) \geq 4 \right]. \quad (4)$$

A six-query cooldown spaces successive assistance events. Until a trigger occurs, $\pi_{0.5}$ continues executing its proposals directly. Figure 1 illustrates the local prediction and Astra recovery paths.

Upon handoff, the host clears the remaining $\pi_{0.5}$ action queue and transfers environment control to Astra. Astra receives external and wrist RGB images, the task instruction, measured robot state, the policy proposal, and execution history. It can either issue an end-effector (EEF) correction or return control to $\pi_{0.5}$. The host validates the response and queries a fresh policy proposal before resuming normal execution.

2.4 Astra takeover and recovery flow

During takeover, Astra evaluates the current visual state, previous execution, and the policy proposal. If a correction is necessary, it outputs an EEF chunk containing 30–50 sequential one-step corrections. Each accepted correction advances one LIBERO environment step. After executing the chunk, the host refreshes the observation and policy proposal, allowing Astra to either continue recovery or relinquish control.

The EEF adapter maps position, rotation, and gripper commands to LIBERO’s seven-dimensional action interface using the operational-space controller’s scales and pose conventions. The host validates commands, executes actions, checks task completion, and manages termination. All recovery occurs within the same environment, allowing $\pi_{0.5}$ to seamlessly resume as the default controller.

3 Experiments

3.1 Evaluation and measures

We evaluate our method on the LIBERO-10, Object, Goal, and Spatial suites, testing ten tasks per suite and five episodes per task. Both pure $\pi_{0.5}$ and the hybrid system complete 200 evaluation episodes. We record task success, handoff episodes, episodes containing Astra actions, Astra step share, token usage, and environment-action throughput.

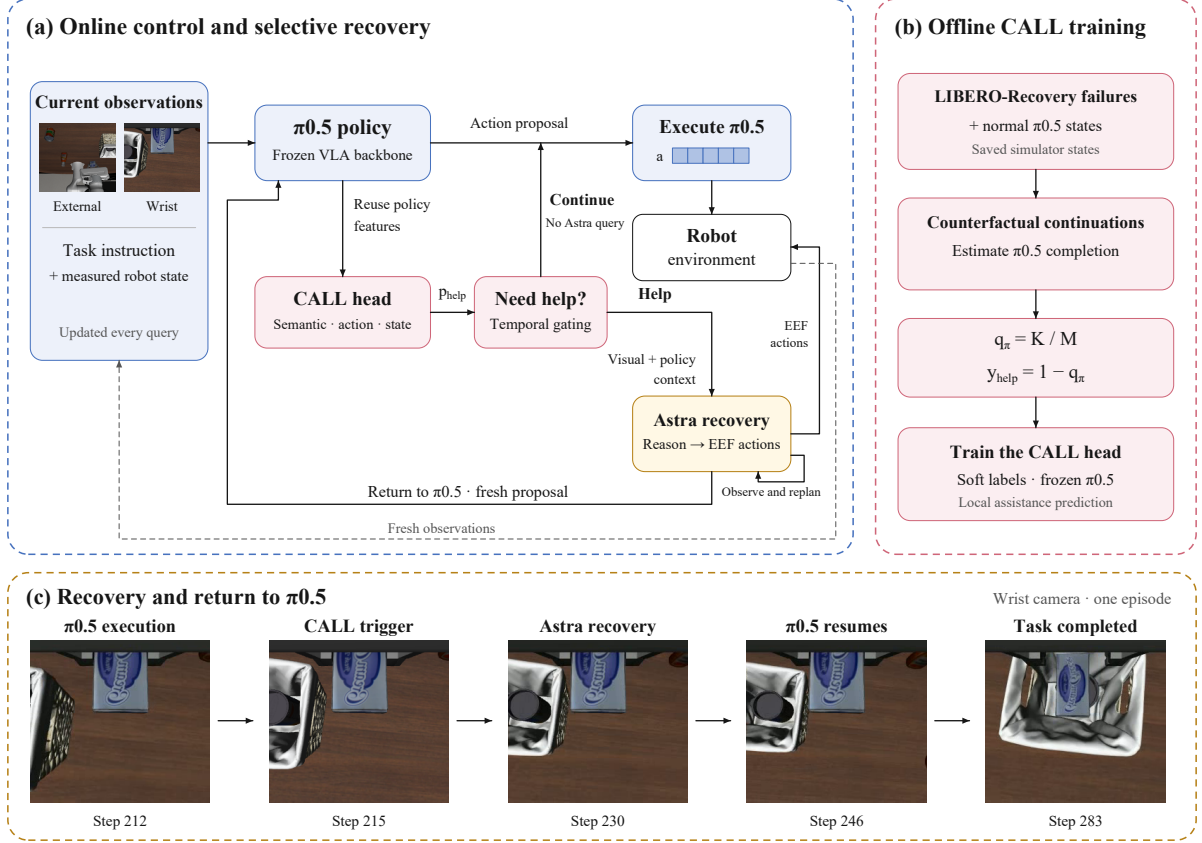


Figure 1: Method overview. (a) CALL reuses frozen $\pi_{0.5}$ features to predict when help is needed. Normal execution proceeds without an Astra query; selected requests activate Astra recovery, followed by a fresh $\pi_{0.5}$ proposal when control returns. (b) Counterfactual $\pi_{0.5}$ continuations provide soft help labels for CALL training. (c) Real wrist-camera frames show execution, a CALL trigger, Astra recovery, resumed $\pi_{0.5}$ control, and task completion within one episode.

Let A_i , E_i , and T_i denote Astra actions, total environment steps, and elapsed seconds in episode i . For an episode set $\mathcal{S}$, Astra step share and action throughput are

$$r_{\text{Astra}} = \frac{\sum_i A_i}{\sum_i E_i}, \quad v(\mathcal{S}) = \frac{\sum_{i \in \mathcal{S}} E_i}{\sum_{i \in \mathcal{S}} T_i} \text{ steps/s.} \quad (5)$$

Throughput is calculated as total environment steps divided by total elapsed time. Tokens per test episode are calculated as the suite token total divided by its 50 episodes. A handoff episode contains an Astra consultation; an action episode contains at least one executed Astra action.

3.2 Task success

The hybrid system achieves a 97.5% overall success rate (195/200 episodes), improving upon the 95.5% baseline (191/200) of pure $\pi_{0.5}$. Table 3 and Figure 2 detail the suite-level results.

Table 3: Success over 50 episodes per suite and 200 overall. Differences are percentage points (pp).

Suite	Pure $\pi_{0.5}$	Astra + $\pi_{0.5}$	Difference
LIBERO-10	43/50 (86%)	47/50 (94%)	+8 pp
Object	50/50 (100%)	50/50 (100%)	0 pp
Goal	49/50 (98%)	49/50 (98%)	0 pp
Spatial	49/50 (98%)	49/50 (98%)	0 pp
Total	191/200 (95.5%)	195/200 (97.5%)	+2 pp

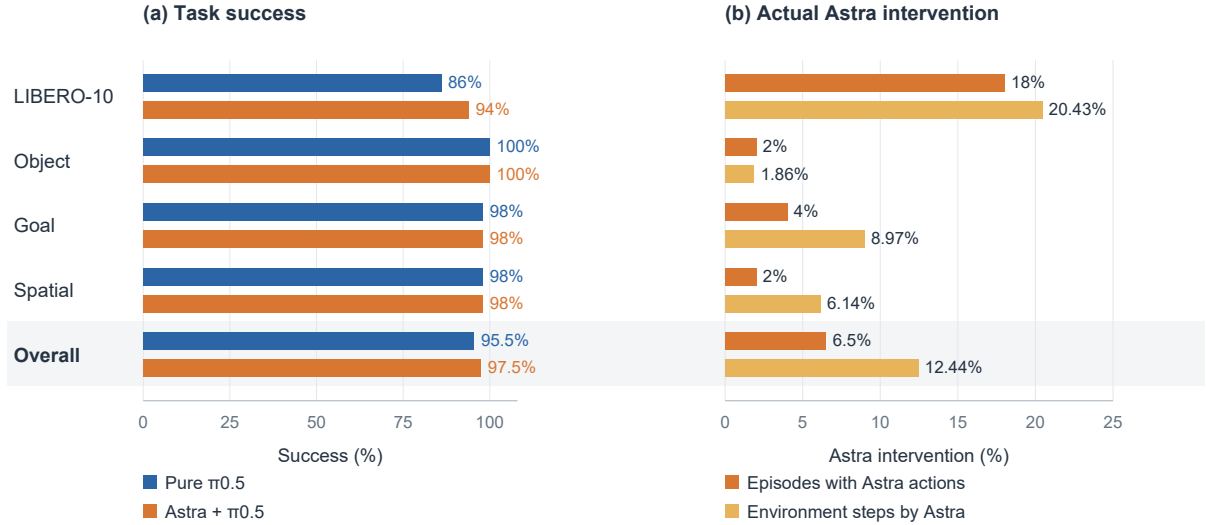


Figure 2: Task success and actual Astra intervention across four LIBERO suites and overall. (a) Success rates of pure $\pi_{0.5}$ and the hybrid system. (b) Percentage of test episodes containing executed Astra actions and percentage of environment steps executed by Astra.

3.3 Selective consultation and actual intervention

The system consults Astra in 61/200 episodes (30.5%) and executes Astra actions in 13/200 episodes (6.5%). In 48 episodes, Astra returns control to $\pi_{0.5}$ immediately after consultation. Across all evaluations, Astra executes 4,561 of 36,654 environment steps (12.44%), leaving $\pi_{0.5}$ to handle the remaining 87.56%.

Across the Object, Goal, and Spatial suites, Astra actions occur in only 4/150 episodes (2.67%), accounting for 1,076/19,592 environment steps (5.49%). The vast majority of episodes and steps remain on the ordinary policy path. Table 4 and Figure 2 summarize these intervention metrics.

Table 4: Astra handoff episodes, action episodes, and environment-step share.

Suite	Handoff ep.	Action ep.	Astra steps	Step share
LIBERO-10	31/50	9/50	3,485	20.43%
Object	19/50	1/50	132	1.86%
Goal	8/50	2/50	560	8.97%
Spatial	3/50	1/50	384	6.14%
Total	61/200	13/200	4,561	12.44%

3.4 Success at assistance states

We evaluate $\pi_{0.5}$ continuation from states at which CALL requests assistance and states immediately before Astra executes its first recovery action. Each saved state is continued five times using the remaining episode-step budget, without an Astra query. The evaluation contains 59 first-CALL states and 13 pre-intervention states. Six states are shared between the two groups, yielding 66 distinct states and 330 continuation trials. Table 5 combines these continuation results with the success rates of episodes without consultation or executed Astra actions.

Table 5: Success without assistance and pure $\pi_{0.5}$ continuation success at assistance states. Each selected state has five continuation trials.

Evaluation subset	Episodes or states	Success rate
Episodes without Astra consultation	139 episodes	139/139 (100%)
Episodes without executed Astra actions	187 episodes	187/187 (100%)
Pure $\pi_{0.5}$ at first CALL states	59 states	229/295 (77.63%)
Pure $\pi_{0.5}$ before actual Astra intervention	13 states	33/65 (50.77%)

Success is substantially lower when CALL is triggered and lower still immediately before actual Astra intervention. Episodes without consultation or executed Astra actions achieve 100% success, whereas pure $\pi_{0.5}$ continuation succeeds in 77.63% of trials at first-CALL states and 50.77% before actual intervention. These observations show that assistance is requested at states where ordinary policy execution is less reliable.

Table 6: Pure $\pi_{0.5}$ continuation success across suites. The CALL and pre-intervention columns use the selected states in Table 5.

Suite	At first CALL states	Before actual Astra intervention
LIBERO-10	103/150 (68.67%)	25/45 (55.56%)
Object	91/95 (95.79%)	5/5 (100%)
Goal	25/40 (62.50%)	3/10 (30%)
Spatial	10/10 (100%)	0/5 (0%)
Total	229/295 (77.63%)	33/65 (50.77%)

Table 6 shows that assistance-state success varies with task difficulty. On the more challenging LIBERO-10 suite, the consultation rate reaches 62%, compared with 16% on Goal and 6% on

Spatial. Pure $\pi_{0.5}$ continuation on LIBERO-10 succeeds in only 68.67% of trials at first-CALL states and 55.56% before actual intervention. Difficult tasks therefore exhibit more frequent assistance requests and lower success when assistance is triggered.

The Goal suite performs particularly well, achieving 98% overall success with consultations in only 16% of episodes. Pure $\pi_{0.5}$ continuation succeeds in 62.50% of trials at first-CALL states and 30% before actual intervention, concentrating assistance on states that are difficult for the base policy. Object, in contrast, has a 38% consultation rate despite 95.79% continuation success at first-CALL states, indicating frequent unnecessary triggers. This behavior likely reflects the comparatively low difficulty and homogeneous scenes of Object, together with an overrepresentation of failure states in its training data. Future work will refine the CALL module and rebalance its training data to reduce unnecessary assistance requests. Task-level episode and continuation results are provided in Appendix A.

3.5 Token expenditure

Table 7 reports input, output, total, and per-episode Astra token expenditure. In aggregate, the Object, Goal, and Spatial suites consume 5,026.97 tokens per test episode while maintaining near-perfect success rates. Figure 3 illustrates per-episode usage.

Table 7: Astra token expenditure across suites.

Suite	Input tokens	Output tokens	Total tokens	Tokens/test episode
LIBERO-10	2,278,625	143,566	2,422,191	48,443.82
Object	93,608	5,677	99,285	1,985.70
Goal	375,366	24,869	400,235	8,004.70
Spatial	238,278	16,248	254,526	5,090.52
Total	2,985,877	190,360	3,176,237	15,881.19

Total Astra usage comprises 2,985,877 input and 190,360 output tokens. Input tokens convey camera observations, task instructions, measured robot state, policy proposals, execution history, and the recovery-tool interface; 2,346,403 input tokens reuse context through prompt caching. Output tokens support embodied reasoning, recovery-action commands, and decisions to return control to $\pi_{0.5}$, including 35,648 reasoning tokens. Because CALL detection runs locally, token expenditure is strictly limited to active Astra consultations.

3.6 Action throughput

Table 8 compares pure $\pi_{0.5}$ throughput with hybrid episodes that proceed without Astra handoff, as well as the overall throughput across all logged episodes. CALL remains active throughout the 139 no-handoff episodes analyzed here.

The no-handoff path preserves the baseline execution speed across all four suites. For example, Goal reaches 8.25 steps/s with CALL active versus 8.17 for pure $\pi_{0.5}$, and Spatial reaches 6.94 versus 6.79 steps/s. This confirms that CALL successfully operates without reducing normal $\pi_{0.5}$ inference throughput.

Across all recorded hybrid episodes, overall throughput drops to 2.58 steps/s due to the latency of Astra consultation and recovery. This division of work ensures $\pi_{0.5}$ maintains fast execution globally, restricting time-intensive reasoning strictly to necessary recovery events. Figure 3 summarizes the throughput comparisons.

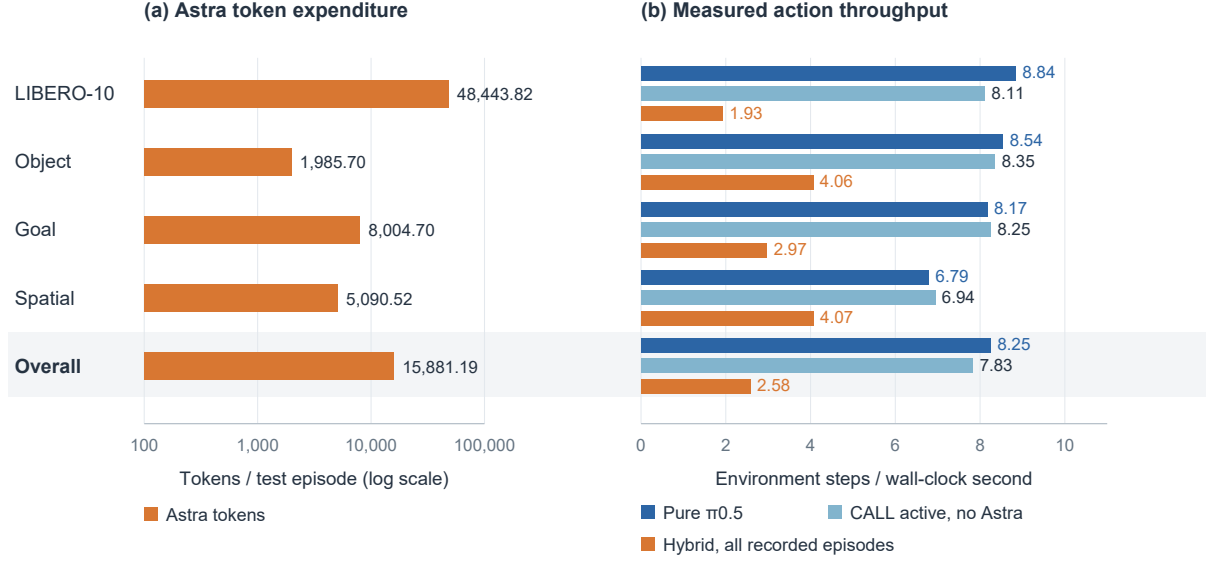


Figure 3: Token expenditure and measured action throughput. (a) Astra tokens per test episode on a logarithmic scale. (b) Environment actions per wall-clock second for pure $\pi_{0.5}$, hybrid episodes with CALL active and no Astra handoff, and all recorded hybrid episodes. Overall throughput is computed from summed steps divided by summed elapsed time.

Table 8: Measured action throughput in environment steps per wall-clock second.

Suite	Pure $\pi_{0.5}$	Hybrid	
		no handoff	all logged episodes
LIBERO-10	8.84	8.11	1.93
Object	8.54	8.35	4.06
Goal	8.17	8.25	2.97
Spatial	6.79	6.94	4.07
Overall	8.25	7.83	2.58

4 Conclusion

We equip $\pi_{0.5}$ with a learned predictor to locally determine when assistance is necessary. CALL utilizes the policy’s existing representations, avoiding external model queries during normal execution. A selective handoff activates Astra for closed-loop recovery, and a return-control mechanism allows $\pi_{0.5}$ to seamlessly resume operation once the state is stabilized.

The system achieves a 97.5% success rate across 200 LIBERO episodes. In the Object, Goal, and Spatial suites, it maintains 100%, 98%, and 98% success while allocating only 5.49% of environment steps to Astra and consuming approximately 5,027 tokens per episode. Furthermore, CALL successfully operates without reducing the inference throughput of the base $\pi_{0.5}$ model. The resulting framework combines fast everyday execution with the advanced reasoning capabilities of Astra precisely when recovery is required.

References

- [1] A. Menon, S. Machcha, S. Zou, T. K. Chan, and J. Chooi. *GPT-6 Astra on Robotic Manipulation*. RoboCurve, 4 September 2026. <https://openai.robocurve.org/gpt-6-astra/>.
- [2] J. Su, Y. Zheng, M. Yan, L. Yi, Z. Zhang, and H. Wang. *GPT 6 Astra as an Embodied Policy*. Technical report and code, 2026. <https://github.com/anonymous-report-421/GPT-as-Policy>.
- [3] Physical Intelligence et al. $\pi_{0.5}$: *A Vision-Language-Action Model with Open-World Generalization*. arXiv:2504.16054, 2025. <https://arxiv.org/abs/2504.16054>.
- [4] Physical Intelligence. *OpenPI: Open-source robotics models and packages*. <https://github.com/Physical-Intelligence/openpi>. Accessed 3 October 2026.
- [5] L. Liu, Z. Bao, L. Zhang, Z. Song, W. Yang, Y. Zhuang, S. Tao, W. Liu, C. Jia, and H. Lu. *LIBERO-RECOVER: Beyond Task Success Towards Failure Recovery in Robotic Manipulation Models*. arXiv:2609.05178, 2026. <https://arxiv.org/abs/2609.05178>.
- [6] LIBERO-Recovery. *Dataset assets, pre-failure histories, and benchmark code*. <https://github.com/liulin815/LIBERO-Recovery>. Accessed 3 October 2026.
- [7] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. *LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning*. NeurIPS 2023. <https://arxiv.org/abs/2306.03310>.

A Task-level evaluation results

A.1 Task-level results

Table A1 presents success and intervention metrics for all 40 tasks. Task indices start at zero within each suite. **Fold** identifies a task-specific CALL head, and **Fallback** indicates a suite-level head.

Table A1: Task-level success and selective intervention.

Suite	Task	Success	Handoff	Action ep.	Astra steps	Head
LIBERO-10	0	4/5	1	1	480	Fold
LIBERO-10	1	5/5	0	0	0	Fold
LIBERO-10	2	5/5	5	1	30	Fold
LIBERO-10	3	5/5	5	0	0	Fold
LIBERO-10	4	5/5	2	0	0	Fold
LIBERO-10	5	5/5	0	0	0	Fold
LIBERO-10	6	5/5	5	0	0	Fold
LIBERO-10	7	5/5	4	2	212	Fold
LIBERO-10	8	4/5	4	4	2,393	Fold
LIBERO-10	9	4/5	5	1	370	Fold
Object	0	5/5	0	0	0	Fold
Object	1	5/5	0	0	0	Fold
Object	2	5/5	0	0	0	Fold
Object	3	5/5	5	0	0	Fold
Object	4	5/5	0	0	0	Fold
Object	5	5/5	0	0	0	Fold
Object	6	5/5	5	1	132	Fold
Object	7	5/5	1	0	0	Fold
Object	8	5/5	5	0	0	Fold
Object	9	5/5	3	0	0	Fold
Goal	0	5/5	0	0	0	Fold
Goal	1	5/5	0	0	0	Fallback
Goal	2	5/5	0	0	0	Fold
Goal	3	5/5	0	0	0	Fold
Goal	4	5/5	2	1	165	Fold
Goal	5	5/5	2	0	0	Fold
Goal	6	5/5	0	0	0	Fold
Goal	7	5/5	0	0	0	Fallback
Goal	8	5/5	0	0	0	Fold
Goal	9	4/5	4	1	395	Fold
Spatial	0	5/5	0	0	0	Fold
Spatial	1	5/5	0	0	0	Fold
Spatial	2	5/5	1	0	0	Fallback
Spatial	3	5/5	0	0	0	Fold
Spatial	4	5/5	0	0	0	Fold
Spatial	5	5/5	0	0	0	Fold
Spatial	6	5/5	0	0	0	Fold
Spatial	7	5/5	0	0	0	Fold
Spatial	8	4/5	2	1	384	Fold
Spatial	9	5/5	0	0	0	Fold

A.2 Continuation before actual Astra intervention

Table A2 presents pure $\pi_{0.5}$ continuation success by task for the 13 states selected immediately before actual Astra intervention. Each state has five continuation trials.

Table A2: Task-level pure $\pi_{0.5}$ continuation before actual Astra intervention.

Suite	Task	States	Success rate
LIBERO-10	0	1	5/5 (100%)
LIBERO-10	2	1	0/5 (0%)
LIBERO-10	7	2	10/10 (100%)
LIBERO-10	8	4	10/20 (50%)
LIBERO-10	9	1	0/5 (0%)
Object	6	1	5/5 (100%)
Goal	4	1	3/5 (60%)
Goal	9	1	0/5 (0%)
Spatial	8	1	0/5 (0%)
Total	–	13	33/65 (50.77%)

A.3 Continuation at first CALL states

Table A3 presents pure $\pi_{0.5}$ continuation success by task for the 59 first-CALL states. Each state has five continuation trials.

Table A3: Task-level pure $\pi_{0.5}$ continuation at first CALL states.

Suite	Task	States	Success rate
LIBERO-10	2	5	25/25 (100%)
LIBERO-10	3	5	3/25 (12%)
LIBERO-10	4	2	10/10 (100%)
LIBERO-10	6	5	15/25 (60%)
LIBERO-10	7	4	20/20 (100%)
LIBERO-10	8	4	10/20 (50%)
LIBERO-10	9	5	20/25 (80%)
Object	3	5	25/25 (100%)
Object	6	5	25/25 (100%)
Object	7	1	5/5 (100%)
Object	8	5	25/25 (100%)
Object	9	3	11/15 (73.33%)
Goal	4	2	7/10 (70%)
Goal	5	2	9/10 (90%)
Goal	9	4	9/20 (45%)
Spatial	2	1	5/5 (100%)
Spatial	8	1	5/5 (100%)
Total	–	59	229/295 (77.63%)